Passive Liveness (PL) API Documentation

OVERVIEW

The Passive Liveness (PL) API provides screen replay detection for identity and liveness workflows. The system analyzes a submitted image and determines whether it is likely a photo of a screen (for example, a face shown on a phone, tablet, laptop, monitor, or TV) rather than a direct capture of a real person.

The PL API validates images through an internal detection pipeline and returns a pass/fail decision plus an overall confidence score. Per-cue metrics, fusion logic, and device classification are not exposed in API responses.

There is a rate limit because we do not want miners spamming the PL API. We want carefully thought-out Unknown Attack Vectors (UAVs) that can attack our system.

NOTE: Only miners with registered hotkeys on the subnet can call the PL API. Static validator hotkeys are always allowed; registered miner hotkeys are fetched from the chain and refreshed every 24 hours.

AUTHENTICATION

All API requests require authentication using hotkey-based signature verification. You must include a signed message in your request that verifies your hotkey identity.

Rate Limiting:

- One request per hotkey every 5 minutes (300 seconds)

- Rate limits are enforced per hotkey address
- Use the /rate-limit/status endpoint to check your current status

API ENDPOINTS

1. POST /validate
   Validate a single miner address

```
import base64
 import requests
 from datetime import datetime
 import bittensor

# Miner information
 wallet_name = "miner"
 wallet_hotkey = "m"

# Path to the image to validate
 image_path = "capture.png"

api_url = "http://98.90.28.118:5001/validate"


def sign_message(wallet):
    """Generate a signed message for API authentication."""
    t = datetime.now()
    msg = f"On {t} {t.astimezone().tzname()} API Request"

   sig = wallet.hotkey.sign(data=msg)

   return (
       f"{msg}\n"
       f"\tSigned by: {wallet.hotkey.ss58_address}\n"
       f"\tSignature: {sig.hex()}"
    )


def encode_image(path):
    """Read an image file and return base64-encoded bytes."""
    with open(path, "rb") as f:
        return base64.b64encode(f.read()).decode("utf-8")


# Load wallet
 wallet = bittensor.Wallet(name=wallet_name, hotkey=wallet_hotkey)
```

```
# Build payload
 payload = {
    "image": encode_image(image_path),
    "signature": sign_message(wallet),
 }

# Send request
 response = requests.post(api_url, json=payload)

print(response.status_code)
 print(response.json()
```

Rate Limiting:

The API implements rate limiting to ensure fair usage and system stability. Each hotkey is rate limited to **one request every 5 minutes** (300 seconds). Rate limiting is enforced per hotkey address, meaning each authenticated hotkey has its own independent rate limit counter. If a hotkey exceeds the rate limit, the request will be rejected with a 429 error.

REQUEST FORMAT

For Single Address Validation (/validate):

Required Fields:

- signature: A signed message block from your hotkey
- Image: Base64-encoded image bytes (PNG, JPEG, etc.). A prefix is optional and will be stripped automatically.

Example Request (Single Address):

```
{
   "signature": "<Bytes>On 2025-01-01 CST My test
message</Bytes>\n\tSigned by: YOUR_HOTKEY\n\tSignature:
YOUR_SIGNATURE",
   "image": "iVBORw0KGgoAAAANSUhEUgAA..."
 }
```

RESPONSE FORMAT

Field	Type	Meaning
is_screen_replay	boolean	true if the image is classified as a likely screen replay; false otherwise. Use this for pass/fail logic.
overall_confidence	number	Confidence score in [0, 1]. 0.0 when no screen replay is detected.

Example (screen replay detected):

```
{
  "is_screen_replay": true,
  "overall_confidence": 0.785
}
```

Example (not a screen replay):

```
{
  "is_screen_replay": false,
  "overall_confidence": 0.0
}
```

PROCESS OVERVIEW

When you submit an address to the LDS system, the following high-level process occurs:

1. **Authentication** — Your request is authenticated using hotkey signature verification.
2. **Rate limiting** — Quota is checked (one /validate call per hotkey every 5 minutes).
3. **Image ingestion** — Base64 is decoded; the image is validated and converted to RGB PNG.
4. **Detection** — The image is analyzed by the internal screen-replay pipeline.
5. **Response** — You receive is_screen_replay and overall_confidence only.

ERROR RESPONSES

• 400 Bad Request: Missing required fields or invalid request format
• 401 Unauthorized: Signature verification failed
• 403 Forbidden: Hotkey not authorized to use the API
• 429 Too Many Requests: Rate limit exceeded (wait 5 minutes between requests)